



Faculty of Informatics

Department of Computer Science

Windows Programming Laboratory Manual

(Student Version)

Prepared By: Mr. Amsalu Dinote (MSc)

November 16 2020

Hawassa, Ethiopia

Windows Programming Laboratory Manual

Objectives of the Manual

At the end of the laboratory exercises the students will be able to

- Write C# programs using any code editor such as Notepad, Notepad++, etc.
- Compiling and executing C# programs from command-line prompt
- Install Microsoft visual studio as an IDE for developing .NET Framework based applications
- Create and execute a C# console application
- Exercise different C# commands on console project
- Create and execute C# Windows Desktop application
- Design GUI for Windows Desktop application using different controls
- Install Microsoft SQL Server
- Create database on Microsoft SQL Server
- Connect windows desktop application with MS SQL Server Database
- Design crystal report using Microsoft visual studio
- Deploy Windows application

Tools required for the Lab

In order to completely and successfully exercise all lab exercises included in this lab manual the following tools are required to be installed on students' computer prior to the start of the lab.

- Microsoft Visual Studio 2012 and above
- Microsoft SQL Server 2014 and above

Instruction for the Student: please, try to struggle for doing the exercises by using your own problem solving skill before looking for answers from your instructor or somebody else. This is very important for developing your own programming skill.

Part I: Introduction to C# programming

What is C# programming?

C# (pronounced as 'C sharp') is a new computer-programming language developed by MS Corporation, USA. It is a fully object-oriented language like Java and is the first component-oriented language. It has been designed to support the key features of .NET Framework, the new development platform of MS for building component-based software solutions. C# is simple, efficient, productive, and type-safe language derived from C and C++ languages. C# is purely object-oriented modern

language suitable for developing web based applications. It is designed for building robust, reliable, and durable components to handle real-world applications

C# borrows Java's features such as grouping of classes, interfaces, and implementations together in one file so that programmers can edit the code more easily. It also handles objects using references, the same way as Java. C# uses VB's approach to form design, namely, dragging controls from toolbox, dropping them onto forms, and writing event handlers for them.

C# has some interesting characteristics:

- **Simple:** simplifies C++ by eliminating annoying operators such as `->`, `::` and pointers.
- **Consistent:** supports a unified type system which eliminates the problem of varying ranges of integer types.
- **Modern:** b/c it supports automatic garbage collection, rich intrinsic model for error handling, decimal data type for financial applications, modern approach for debugging, and robust security model.
- **Type-safe:** incorporates type-safety measures such as all dynamically allocated objects and arrays are initialized to zero, use any uninitialized variables produces error message by the compiler, access to arrays are range-checked, it doesn't permit unsafe casts, enforces overflow checking in arithmetic operations, reference parameters that are passed are type-safe, support automatic garbage collection.
- **Versionable:** provides support for versioning with the help of *new* and *override* keywords.
- **Compatible:** enforces the .NET common language specifications and therefore allows interoperation with other .NET languages.
- **Interoperable:** provides support for using COM objects, no matter what language was used to author them.
- **Flexible:** although it doesn't support pointers, we may declare certain classes and methods as 'unsafe' and then use pointers to manipulate them.
- **Object oriented:** is truly object-oriented, supports encapsulation, inheritance, and polymorphism. Everything is an object; there are no more global methods (functions), constants, and variables.

C# is a programming language important to develop Console applications, Windows applications, Developing Windows controls, Developing ASP.NET projects, Creating Web controls, Providing Web Services, Developing .NET component library, and etc.

Key highlights of the C# programming language are the following:

- Ease of Visual Basic with the power of most C++ features
- Similar programming language syntax for C++ and Java developers
- Support for all CLR data types
- Pass by reference and value for parameters
- Operator overloading (not found in Java)
- Capability to run clearly marked "unsafe" code
- XML-based documentation, using special comments
- Integration with Visual Studio .NET for rapid development

Naming Conventions in C#

By using the correct conventions when naming objects, you ensure that other C# programmers who read your code will easily understand what objects are without having to search your code for their definition.

Item	Convention followed to name the item
Namespaces	Are named using Pascal Case with no underscores. This means the first letter of every word in the name is capitalized. For example: <i>MyNewNamespace</i> .
Assemblies	Named by following the same pascal case naming.
Classes and Structures	Named by following Pascal Case, no underscores or leading "C", "cls", or "I". Classes should not have the same name as the namespace in which they reside.
Exception Classes	Follow class naming conventions, but add Exception to the end of the name.
Interfaces	Follow class naming conventions, but start the name with "I" and capitalize the letter following the "I". Example: <i>IFoo</i> The "I" prefix helps to differentiate between Interfaces and classes and also to avoid name collisions.
Functions	Pascal Case, no underscores except in the event handlers. Try to avoid abbreviations.

Properties and Public Member Variables	Pascal Case, no underscores. Try to avoid abbreviations.
Parameters and Procedure-level Variables	Camel Case. Try to avoid abbreviations. Camel Case is the same as Pascal case, but the first letter of the first word is lowercased.
Class-level Private and Protected Variables	Camel Case with a leading underscore. Always indicate 'Protected' or 'Private' in the declaration.
Form and Controls on Forms	Pascal Case with a prefix that identifies it as being part of the UI instead of a purely coded control.

Activity: open your notepad (or any other editor) and write the following code and save your program as *exercise1.cs* Evaluate the code against the conventions written above. (Note that here you are not expected to execute the code, we'll see how to execute the code using command line)

```
public class Customer
{
    private string _customerName;
    private DateTime _lastUpdated;
    Public string CustomerName
    { get{ return _customerName;}
      set{ _customerName = value;
           _lastUpdated = DateTime.Now;
        }
    }
    public DateTimeLastUpdated
    { get{ return _lastUpdated;}
      set{ _lastUpdated=value; }
    }
    public void UpdateCustomer(string newName)
    {
        if(!newName.Equals(customerName))
        {
            CustomerName=newName;
        }
    }
}
```

Comments in C#

Comments allow inline documentation of source code. The C# compiler ignores comments. Like other high-level programming languages such as Java and C++, three styles of comments are allowed in C#:

Comment type	Used marker
Single-line	//
Multiple-line	Start with /* and end with */
Xml-documentation comment	///

Data Types of C#

Most of the data types supported in C# are similar to that of C-family high-level programming languages such as C, C++ and Java. The following tables show the respective data type categories that are the most known and used by programmers of the language.

Integer Types			
C# Alias	.NET Type	Size (bites)	Range
Sbyte	System.Sbyte	8	-128to127
Byte	System.Byte	8	0 to 255
short	System.Int16	16	-32,768 to 32,767
Ushort	System.UInt16	16	0 to 65,535
Char	System.Char	16	A unicode character of code 0 to 65,535
Int	Syste.Int32	32	-2,147,483,648 to 2,147,483,647
uInt32	System.UInt32	32	0 to 4,294,967,295
Long	System.Int64	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Ulong	System.UInt64	64	

Floating-point				
C# Alias	.NET Type	Size (bites)	Precision	Range
Float	System.Float	32	7 digits	1.5×10^{-45} to 3.4×10^{38}
Double	Sytem.Double	64	15-16 digits	5.0×10^{-324} to 1.7×10^{308}
Decimal	System.Decimal	128	28-29 decimal places	1.0×10^{-28} to 7.9×10^{28}

Other Pre-defined Types			
C# Alias	.NET Type	Size (bites)	Precision
Bool	System.Boolean	32	True/false
Object	System.Object	32/64	Platform dependent(a pointer to an object)
String	System.String	16*length	A Unicode string with no special upper bound

Custom Types

The predefined types can be aggregated and extended into custom types. Custom value types are declared with the **struct** or **enum** keyword. Likewise, custom reference types are declared with the class keyword.

Enumeration

An enumeration is a datatype that enumerates a set of items by assigning to each of them an identifier (a name), while exposing an underlying base type for ordering the elements of the enumeration. The underlying type is *int* by default, but can be any one of the integral types except for char. For example, a custom type of weekday can be defined as follows,

```
enum Weekday{Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};
```

Structures

Structures (keyword *struct*) are light-weight objects. They are mostly used when only a data container is required for a collection of value type variables. *Structs* are similar to classes in that they can have constructors, methods, and even implement interfaces, but there are important differences.

- *Structs* are value types while classes are reference types, which mean they behave differently when passed into methods as parameters.

- *Structs* cannot support inheritance. While *structs* may appear to be limited with their use, they require less memory and can be less expensive if used in the proper way.
- *Structs* always have a default constructor, even if you don't want one. Classes allow you to hide the constructor a way by using the "private" modifier, whereas structures must have one. A struct can, for example, be declared like this:

```
struct Person
{
    public string name;
    public System.DateTime birthDate;
    public int heightInCm;
    public int weightInKg;
}
```

NB: *Structs* are really only used for performance reasons and/or if you intend to it by value. *Structs* work best when holding a total equal to or less than 16 bytes of data. If in doubt, use classes.

The remaining all concepts such as *arrays*, *variables*, *constants*, *statements*, *inheritance*, and *functions* are syntactically written in the same as they are written in Java or C++. Therefore, you are expected to recall the knowledge you gained in your Java or C++ classes related to the respective concepts.

Part II: Tools for C# programming

In order to write C# programs one can use either any code editor such as notepad, notepad++, or etc. and save the program with an extension .cs—this indicates that the file is C# file or an integrated development environment (IDE) such as Microsoft Visual Studio.NET. However, since simple editors such notepad doesn't provide an integrated way of compiling and executing the written programs it is preferable to use IDEs like MS VS.Net. Therefore, for this laboratory work we are going to use MS Visual Studio 2012 to write, compile and execute both console and windows applications.

Microsoft Visual Studio is an integrated development environment (IDE) from Microsoft. It is used to develop computer programs for Microsoft Windows, as well as web sites, web applications and web services. Visual Studio uses Microsoft software development platforms such as Windows API,

Windows Forms, Windows Presentation Foundation, Windows Store and Microsoft Silverlight. It can produce both native code and managed code.

Part III: Writing, Compiling and Executing C# Programs

Just like Java programs, C# programs can be written using simple code editors such as Notepad, Notepad++, etc and then compiled and executed from command-line prompt. However, as mentioned in part II, for the sake of rapid and easy coding, compilation, and execution integrated development environments (IDEs) such as MS Visual Studio are ideal tools.

How to Execute C# program from command line prompt?

C# programs written by using simple code editors can be compiled and executed from command-line prompt. Note that if one is using windows operating system on his/her computer C# compiler is also available inside framework folder depending on the bit of operating system and the version of Dot Net Framework installed during the installation of the windows. Therefore, if you are a user of windows-based computer you already have C# compiler, if not you should download the required Dot Net Framework from Microsoft official website as they are the owner of the product.

If one wants to execute a program written by using non-IDE editor he/she has to first locate the C# compiler on the command line by using its default path. This can be accomplished by the **cd** (change directory) command. The default paths for C# compiler are:

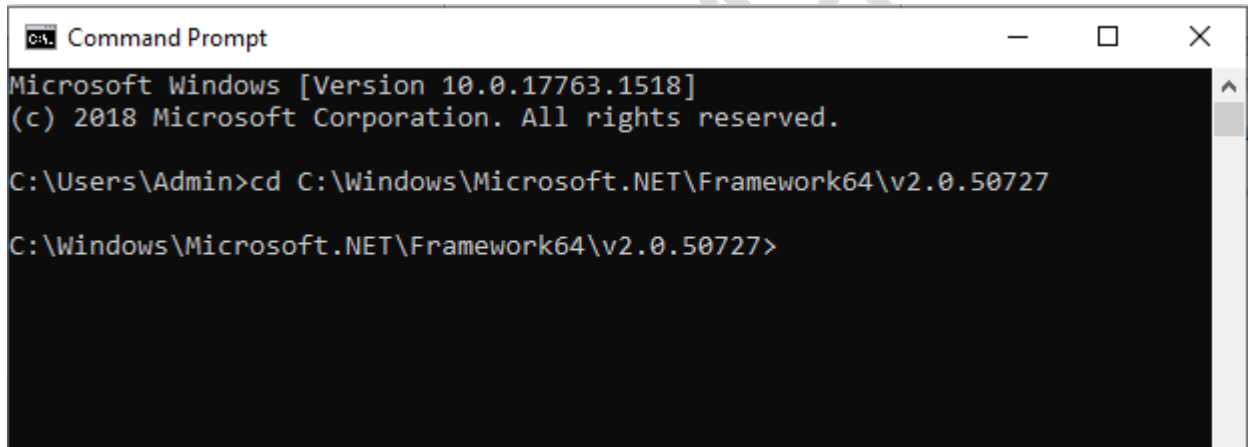
- For .Net framework version 1 at C:\WINDOWS\Microsoft.NET\Framework\v1.0.3705
- For .Net framework version 2 at C:\Windows\Microsoft.NET\Framework64\v2.0.50727
- For .Net framework version 3 at
C:\Windows\Microsoft.NET\Framework64\v3.5
- For .Net framework version 4 at C:\Windows\Microsoft.NET\Framework64\v4.0.30319

Simple steps for executing C# program from command line prompt could include:

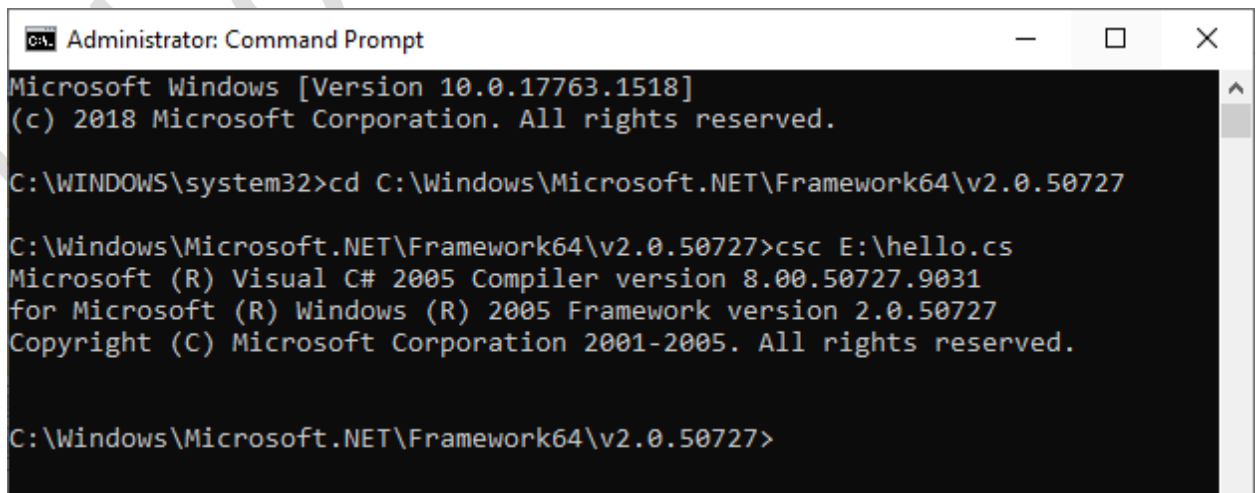
Step 1. Write C# program using notepad or any editor and save the file with the extension .cs. For example, save the following simple program written on Notepad++ with file name **hello.cs**.

```
using System;
public class Hello
{
    public static int Main()
    {
        Console.WriteLine("Hello world");
        return 0;
    }
}
```

- Step 2. Open command line prompt (run as admin to avoid permission-related error) and locate the C# compiler based on the version of Dot Net Framework existing on your operating system using **cd** command. For example, on my operating system the compiler is found in C:\Windows\Microsoft.NET\Framework64\v2.0.50727, hence the **cd** (change directory) command will locate the compiler in this directory.

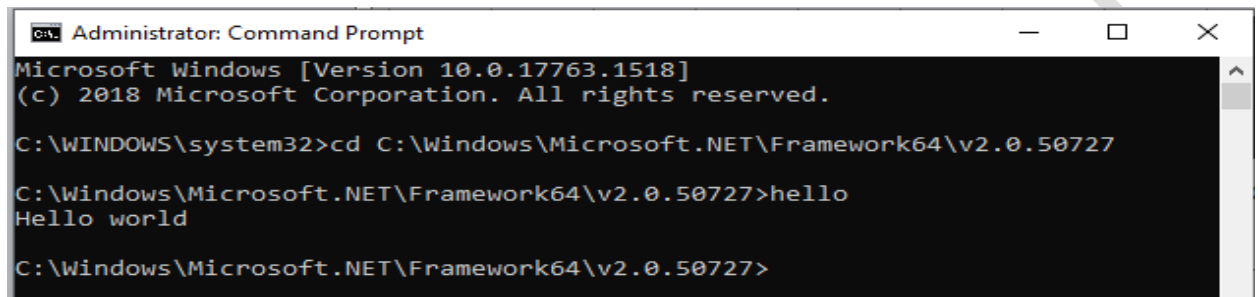


- Step 3. Compile the program using the command **csc**(stands for 'csharp compiler') followed by the path of file and file name with extension. For example, C:\>csc E:\hello.cs



If you observed any compilation errors in the above step try to fix them all, otherwise you will not get the output of the program in the next step. If your program is compilation-errors free, proceed to the next step.

Step 4. Execute the program to see the output, i.e., simply write the name of the file on the command line prompt.



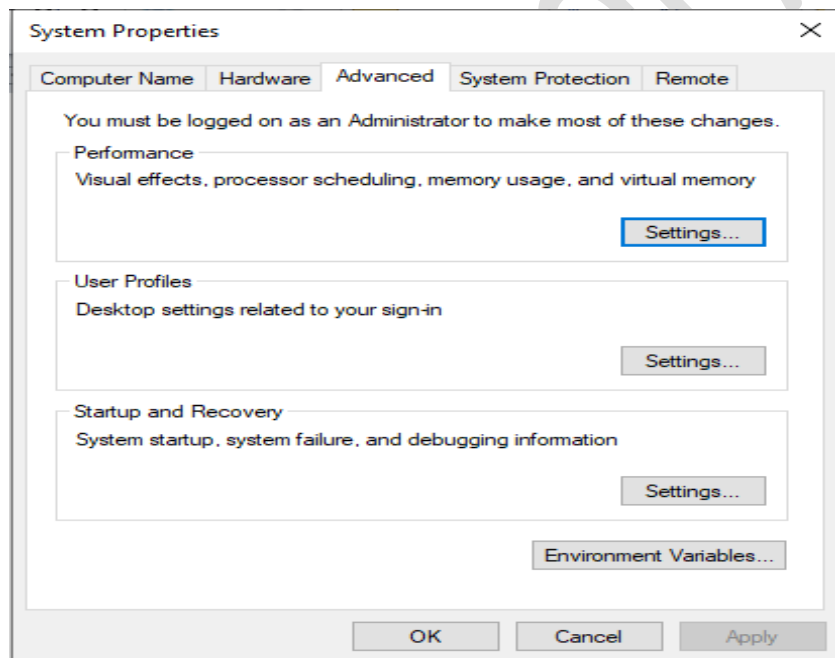
```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.1518]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>cd C:\Windows\Microsoft.NET\Framework64\v2.0.50727

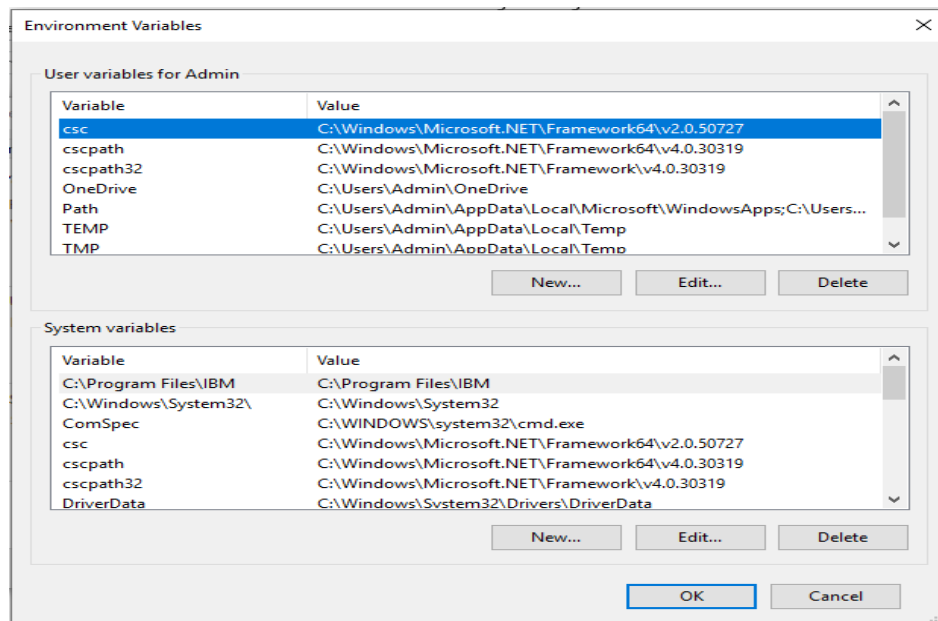
C:\Windows\Microsoft.NET\Framework64\v2.0.50727>hello
Hello world

C:\Windows\Microsoft.NET\Framework64\v2.0.50727>
```

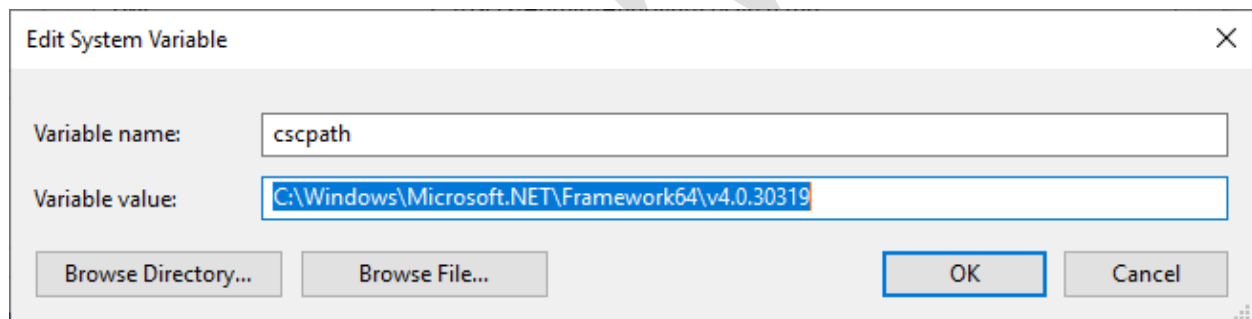
Note that if one is using non-IDE editor every time he/she wants to compile and execute the program she/he has to locate the path of the C# compiler. However, it is possible to avoid such case by registering the path of the compiler in environment variable and calling the compiler from anywhere in file system. To register the path of compiler search for 'Edit the system environment variables' from start menu search box. If you got, the following window will be displayed.



From the above window click on the **Environment Variables...** button, which will display the window shown below.

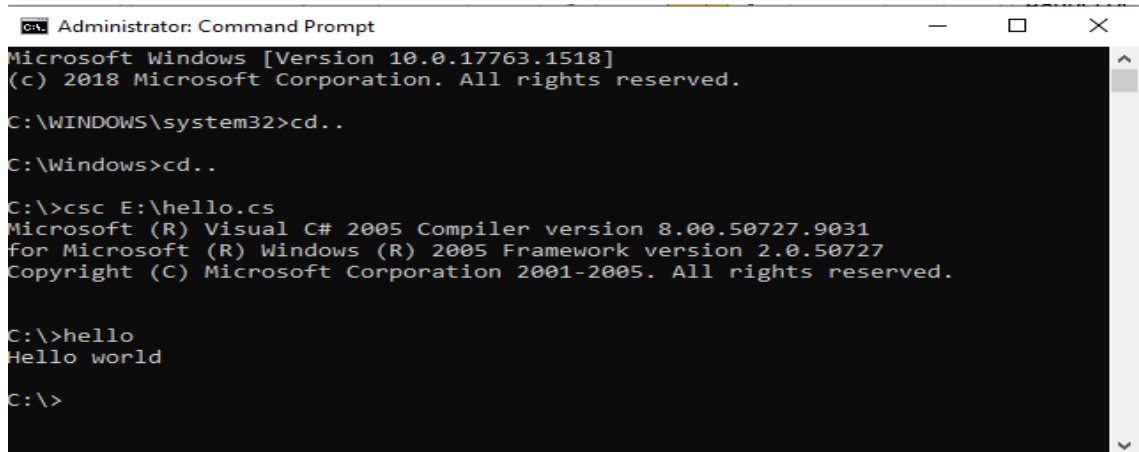


Click on **New...** button under **system variables**. This will display the following windows.



On the above window set the Variable name to any string and in Variable value put the default path of the C# compiler (you can open the folder in which the compiler exists and then copy its address from address). Once you have done with close the windows and rerun command line prompt as admin.

Now, the command `csc` is registered as known command so you can compile and run your program simply write `csc` command followed by the name of the file on the command line prompt without putting the long path of the compiler.



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.1518]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>cd..
C:\Windows>cd..
C:\>csc E:\hello.cs
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.9031
For Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. All rights reserved.

C:\>hello
Hello world

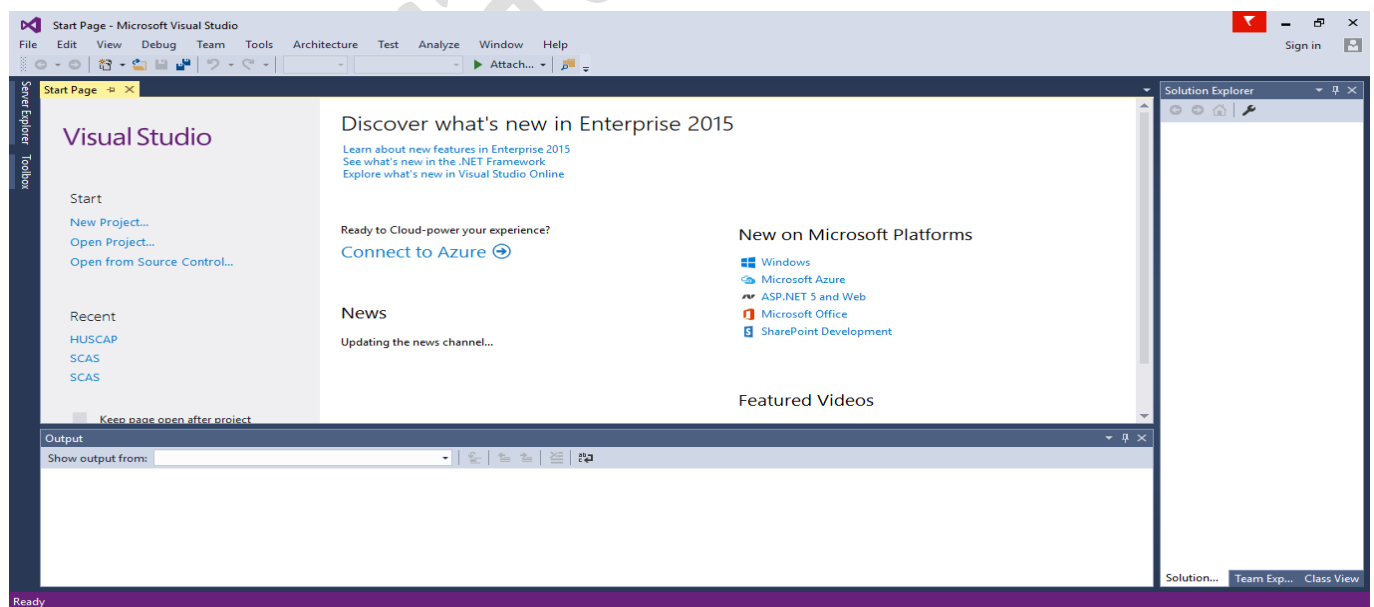
C:\>
```

How to write, compile and execute C# programs using IDEs?

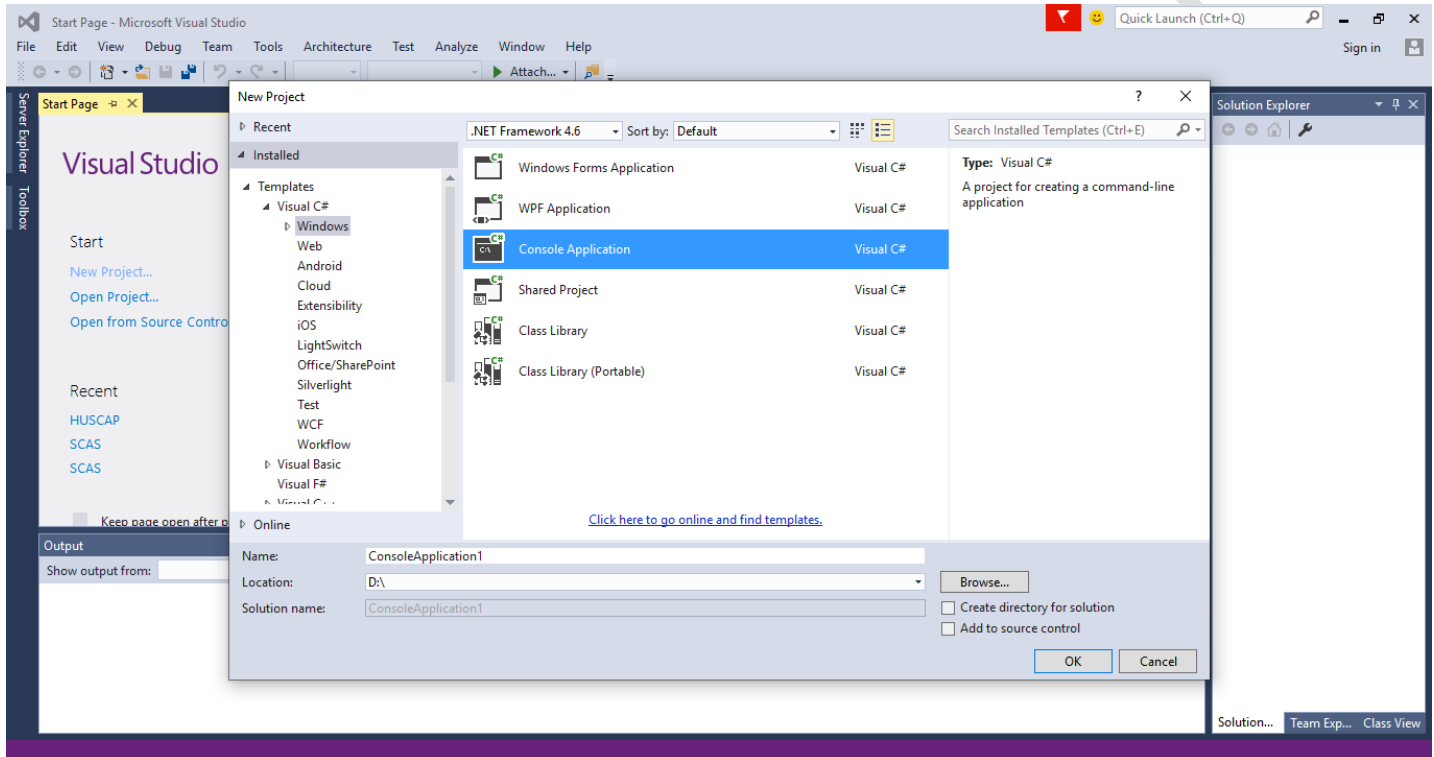
Reasonably, as justified in previous section part II IDEs are very helpful for rapid both console and windows applications development. In this section we are going to create both C# console and windows applications to practice basic programming concepts of C# using MS Visual Studio 2015.

Creating, Writing, Compiling, and Executing Console Application Using MS VS 2015

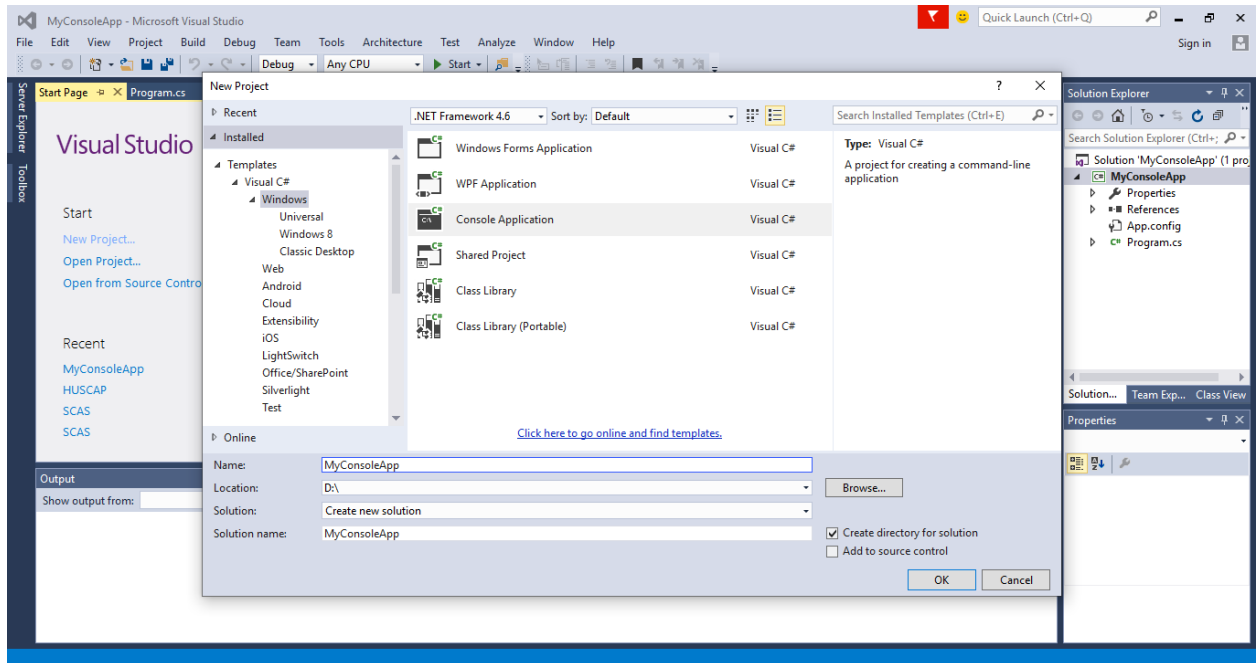
To easily and rapidly create console application using visual studio follow the steps given below.



- Step 1. Open Microsoft Visual Studio from all programs or from taskbar (if available on the taskbar). This action will display the window shown above.
- Step 2. Then Go to **File=>New=>project** or on the start page click on **New Project...** this will display the following window letting you the type of application you are going to create, the name of the application, the location where you want to save your project file.

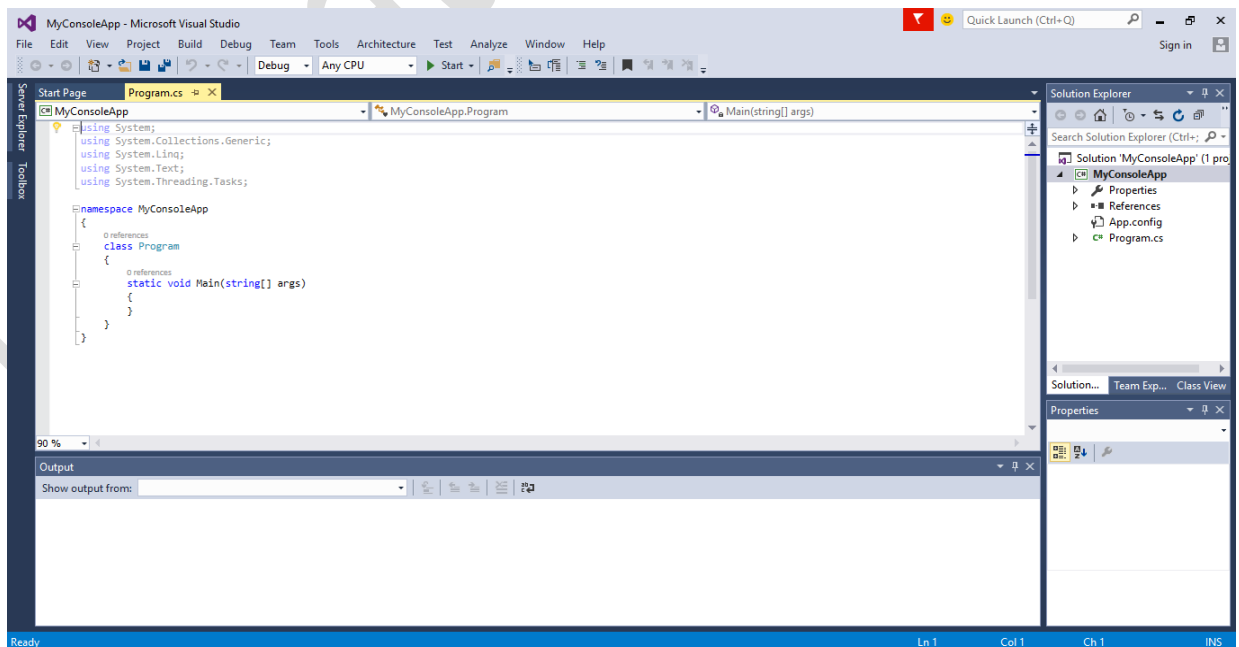


- Step 3. On the left panel click on Installed=>Templates=>Visual C#=>Windows=>Console Application. Then set name of the application and location. Let's say the name of the application is **MyConsoleApp**(prefix by your name like AddisuConsoleApp). Tick the checkbox (this is optional you can tick in you want otherwise leaf it unticked) **Create directory for solution** to let the application created with all its files within a folder named by the name of the application you set during creation. In our case, the folder **MyConsoleApp** folder will be created under local disk D.

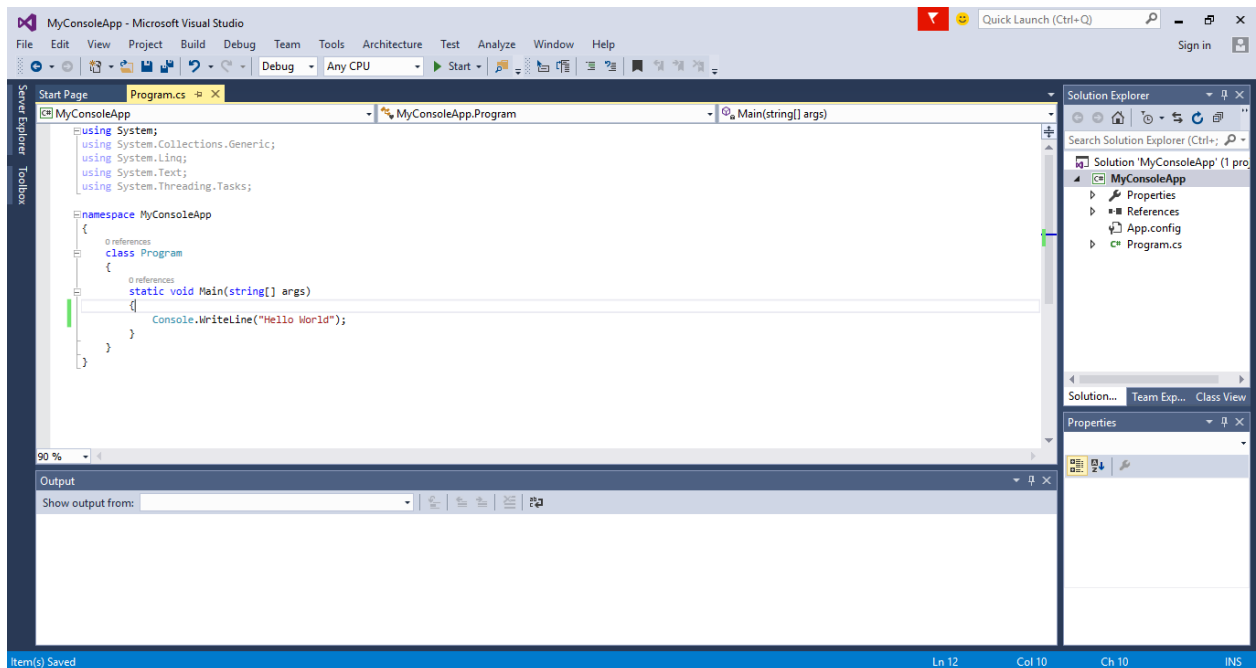


Step 4. If you have correctly done with the previous steps you will get the following window that has the environment to write a required code and on the right side panel under the project name the list of project files created by default.

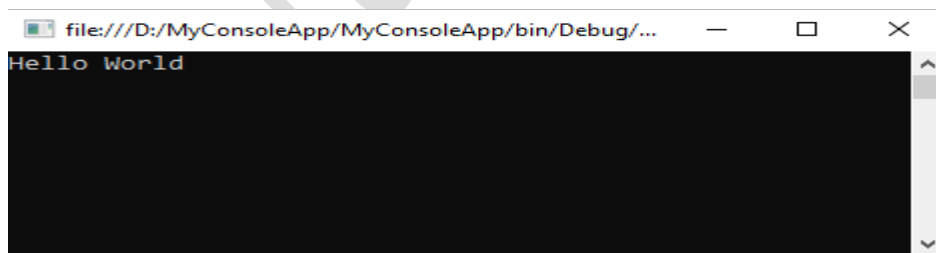
Congratulations!! You created a console application. At this time your application has a namespace with the name of the project and class named **Program** with a **Main** function as its static member function. As you can remember namespace is a collection of related classes and other items, hence, inside the **MyConsoleApp** namespace you can write your own classes



Step 5. Now let's write a C# line of code to print "Hello World" message on the screen. This can be written inside the Main function using **WriteLine()** built-in function from Console class.



Step 6. To compile the project go to Build=>Build Solution(or press Ctrl+Shift+B) and to run the project go to Debug=>Start Debugging(or press F5). Remember that if you run your hello world program you will not see an output screen, because, there should be a call to simple function like ReadKey() which waits for an input to be pressed from keyboard. ReadKey() function is equivalent to the getch() function in C++. Therefore, add this function right after the line Console.WriteLine("Hello World"); and re-run the program.



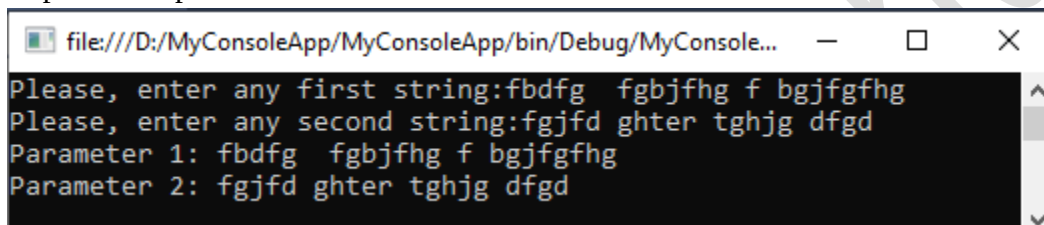
Now you are ready to write, compile and execute more C# codes by your own!!

Exercise Questions on Console Application

Instruction: Inside the console project you have created write the solution of each of the following exercises accordingly.

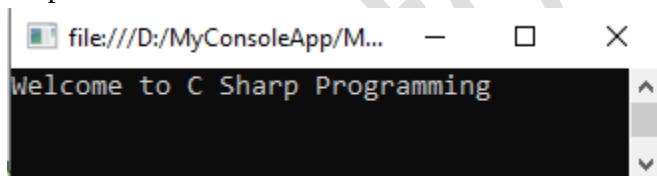
Exercises

1. Write a function named **Question1** that takes two string type parameters and displays the parameters on single line if either of the strings doesn't contains whitespace or the sum of the length of the two strings is less than 50, otherwise each on separate lines. The caller of the function is the main function (the inputs should be read from keyboard). Expected output is shown below.



Answer: (Write your C# code!)

2. Define two classes, **clsC1** and **clsC2**. The first class has a member method named **Display1** that displays the string "C Sharp" and the second class has a member method named **Display2** that displays the string "Programming". Call the methods from within main function to display the output:



Answer: (write your C# code!)

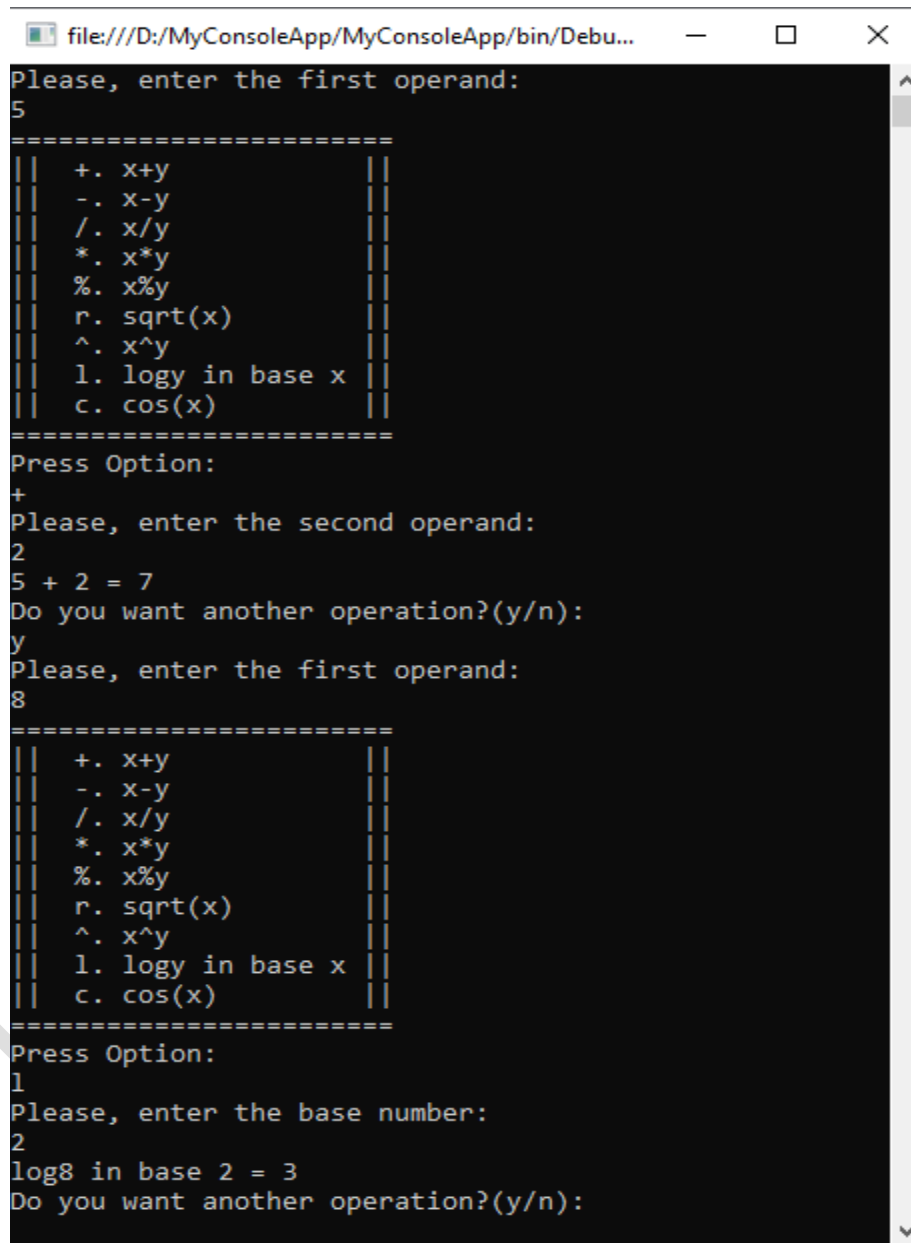
3. write a code inside main function that reads two double type values x and y from keyboard, computes their sum, assigns the result to a third variable z, and displays all the three values in one line of output. **The Sum of 2 and 5 is 7**

Answer: (write your C# code!)

4. Define a class named **SimpleCalculator** that has method to perform basic arithmetic and other mathematical operations on two values. The member methods include:
 - a. **Add** takes the two values and returns the sum
 - b. **Subtract** takes the two values and returns the difference
 - c. **Multiply** takes the two values and returns the product
 - d. **Divide** takes the two values and returns the quotient
 - e. **Module** takes the two values and returns the remainder
 - f. **Root** takes one value and returns the square root

- g. **Power** takes the two values and returns the powered valued, i.e x^y
- h. **Log** takes the two values and returns the logarithm of first value in base second value, i.e $\log_y^x$
- i. **Cos** takes one value and returns the cosine value

Call each of the methods from within main function by passing the required inputs through command-based interface. The expected output screen of your program should look like the following.



```
file:///D:/MyConsoleApp/MyConsoleApp/bin/Debu...
Please, enter the first operand:
5
=====
||  +. x+y      ||
||  -. x-y      ||
||  /. x/y      ||
||  *. x*y      ||
||  %. x%y      ||
||  r. sqrt(x)  ||
||  ^. x^y      ||
||  l. logy in base x ||
||  c. cos(x)   ||
=====
Press Option:
+
Please, enter the second operand:
2
5 + 2 = 7
Do you want another operation?(y/n):
y
Please, enter the first operand:
8
=====
||  +. x+y      ||
||  -. x-y      ||
||  /. x/y      ||
||  *. x*y      ||
||  %. x%y      ||
||  r. sqrt(x)  ||
||  ^. x^y      ||
||  l. logy in base x ||
||  c. cos(x)   ||
=====
Press Option:
1
Please, enter the base number:
2
log8 in base 2 = 3
Do you want another operation?(y/n):
```

Answer: (write your C# code!)

5. Define a structure named **student** that contains a member method named:
- Add** to register new student with personal details such as id, full name, sex, age, department(should only be Computer Science, Information System, Information Technology, Software Engineering, and Data Science), program(regular and CEP)
 - Display** to display all registered students list in tabular form
 - Statistics** that takes department and displays the number of students sex wise, age group wise (16-20, 21-25, 26-30, >30), and program wise in the department.

Expected program features

- ✓ Use concept of properties to set and get the value each member data
- ✓ The program should be able register number of students at a single execution time
- ✓ The program should prevent student with unknown department and/or program from being registered
- ✓ The program should provide command line interface for the user to facilitate the communication

The expected out command-based interface is shown below.

```
file:///D:/MyConsoleApp/MyConsoleApp/bin/Debug/MyConsoleApp.EXE
=====
||  A. Add New Student(s)      ||
||  D. Display Student List   ||
||  S. Department Statistics   ||
=====
Press Option:
a
How many students do you want to add? 3
For Student 0
Please, enter stusent id: cs/0012/08
Please, enter stusent full name: adane kebede gelaye
Please, enter stusent sex: male
Please, enter stusent age: 24
Please, enter stusent department[CS,IS,IT,SE,DS]: cs
Please, enter stusent program[CEP,Regular]: cep
For Student 1
Please, enter stusent id: cs/0031/09
Please, enter stusent full name: aster ammanuel aweke
Please, enter stusent sex: female
Please, enter stusent age: 20
Please, enter stusent department[CS,IS,IT,SE,DS]: cs
Please, enter stusent program[CEP,Regular]: regular
For Student 2
Please, enter stusent id: it/0021/08
Please, enter stusent full name: desta dendeshe desalegn
Please, enter stusent sex: male
Please, enter stusent age: 26
Please, enter stusent department[CS,IS,IT,SE,DS]: it
Please, enter stusent program[CEP,Regular]: regular
Do you want to continue?(y/n)y
```

```

=====
|| A. Add New Student(s) ||
|| D. Display Student List ||
|| S. Department Statistics ||
=====
Press Option:
d
ID          Full Name          Sex          Age          Department          Program
-----
cs/0012/08   adane kebede gelaye         male         24           cs                cep
-----
cs/0031/09   aster ammanuel aweke        female        20           cs                regular
-----
it/0021/08   desta dendesho desalegn     male         26           it                regualr
-----
Do you want to continue?(y/n)y
=====
|| A. Add New Student(s) ||
|| D. Display Student List ||
|| S. Department Statistics ||
=====
Press Option:
s
Please, enter department name: cs
The Statistical Data in cs Department
-----
Male Students:1
Female Students:1
Students of Age Group 16-20:1
Students of Age Group 21-25:1

```

Answer: (write your C# code!)

6. A general quadratic formula for calculating a solution of quadratic equation is given by:

$$X = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

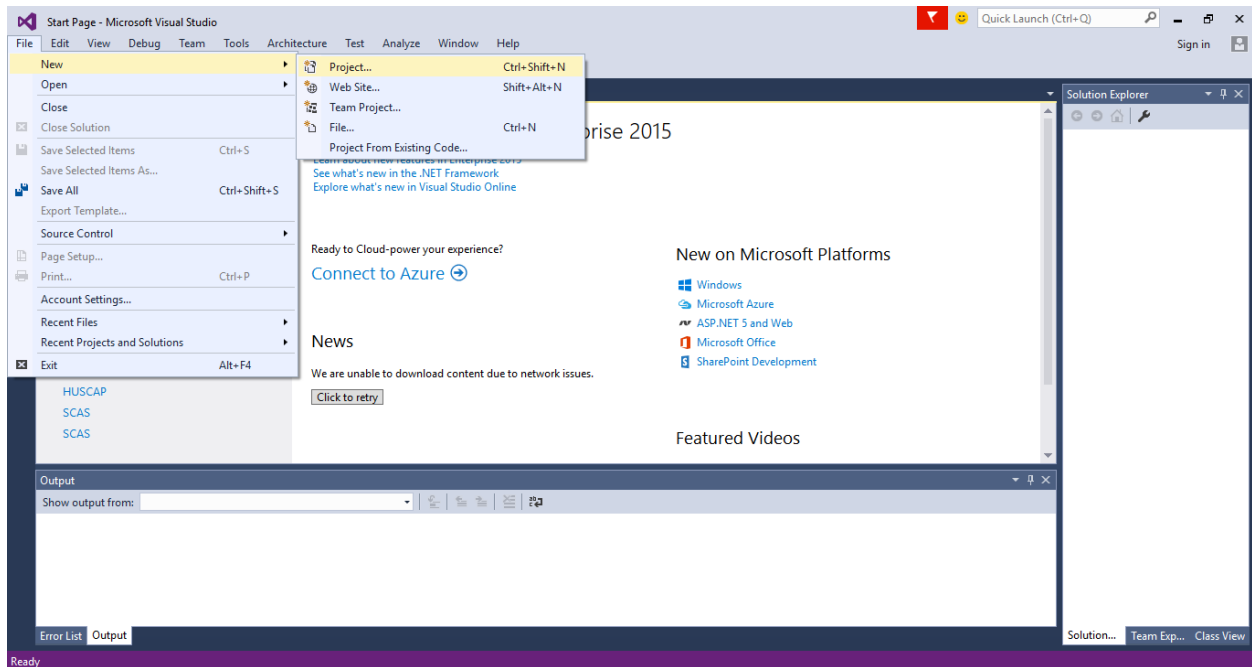
Write a function that calculates the roots of a quadratic function by taking the values of the coefficients a, b, and c from the caller—the main function. A quadratic equation will have real solution if and only if $b^2 - 4ac$ is greater than 0, and only one real root if $b^2 - 4ac$ is equal to 0, and will not have real solution if $b^2 - 4ac$ is less than 0. The function should consider these all situations during computation. (**Work as Exercise**)

Creating, compiling, and executing windows application using MS VS 2015

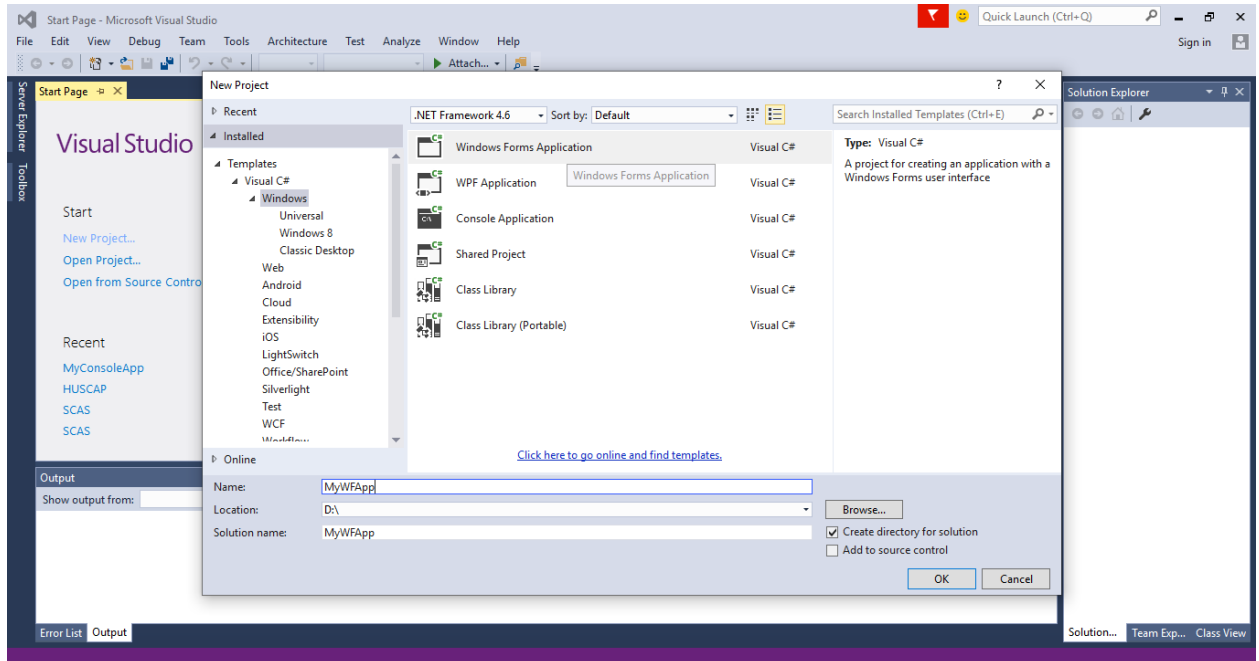
Creating a windows application is similar with the steps we followed in previous session to create console application in MS VS 2015. Anyhow, let's follow the following steps to create windows application.

Step 1. Open MS VS 2015 from Start Menu => All Programs (or click on MS VS icon on taskbar if visual studio is pinned to taskbar)

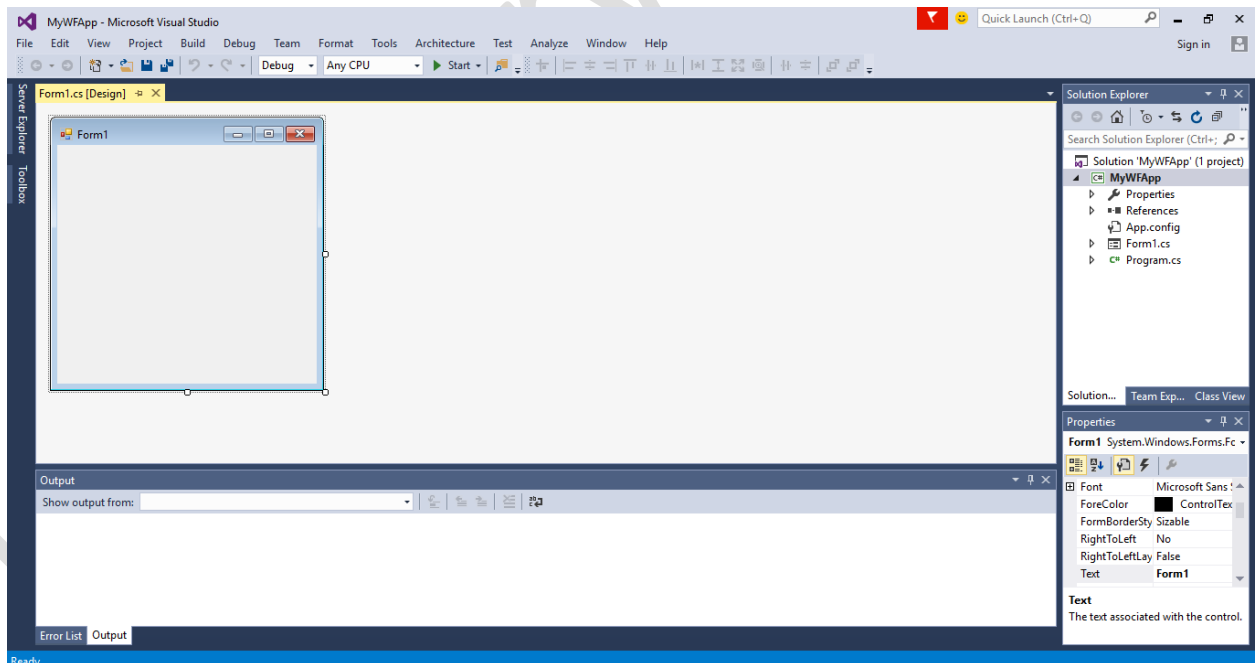
Step 2. Once MS VS opened on start page click on New Project... or go to File=>New=>Project...



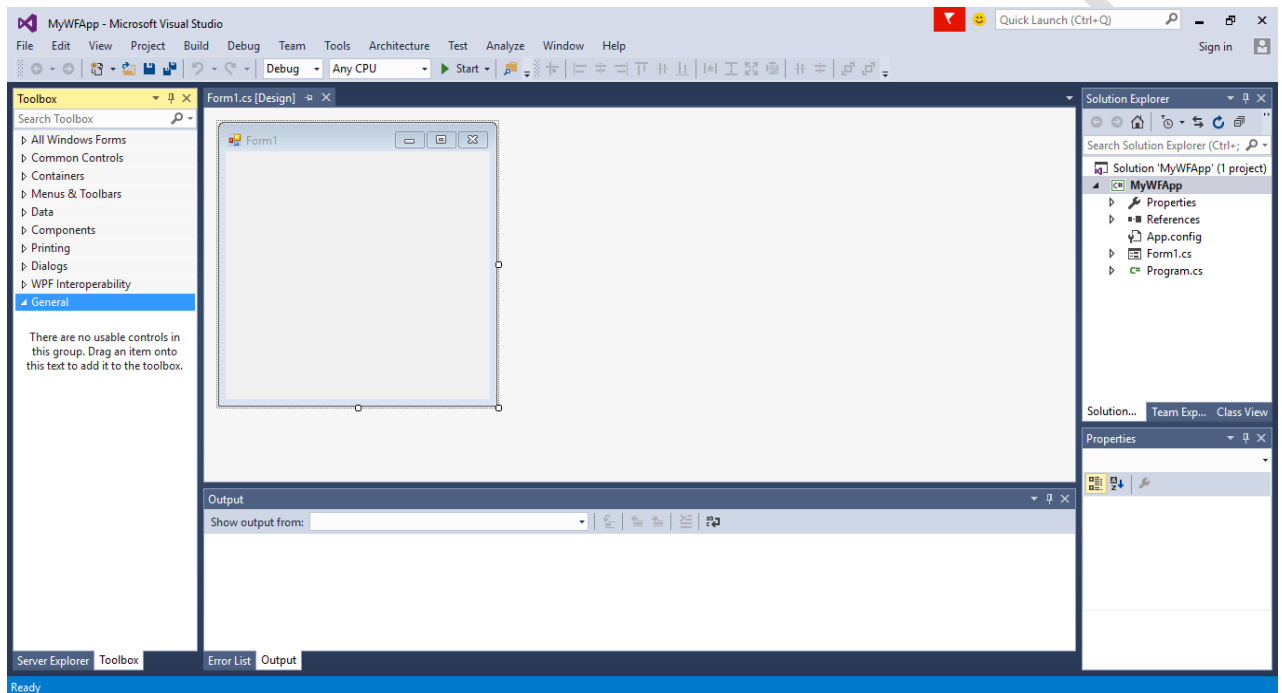
Step 3. Once you have clicked on **New Project...** the New Project windows will be displayed. From that window, on left panel go to **Installed=>Templates=>Windows** and then on the center select **Windows Forms Application**. Set the name and the location of the project. In my case the project name is set to **MyWFApp** and the location is local disk D as shown on the figure below.



Step 4. Click on OK button to create the project. The following window shows the new project with one default form named **Form1**. On the right-side under **Solution Explorer** the project with the name is displayed with the list of all default files.

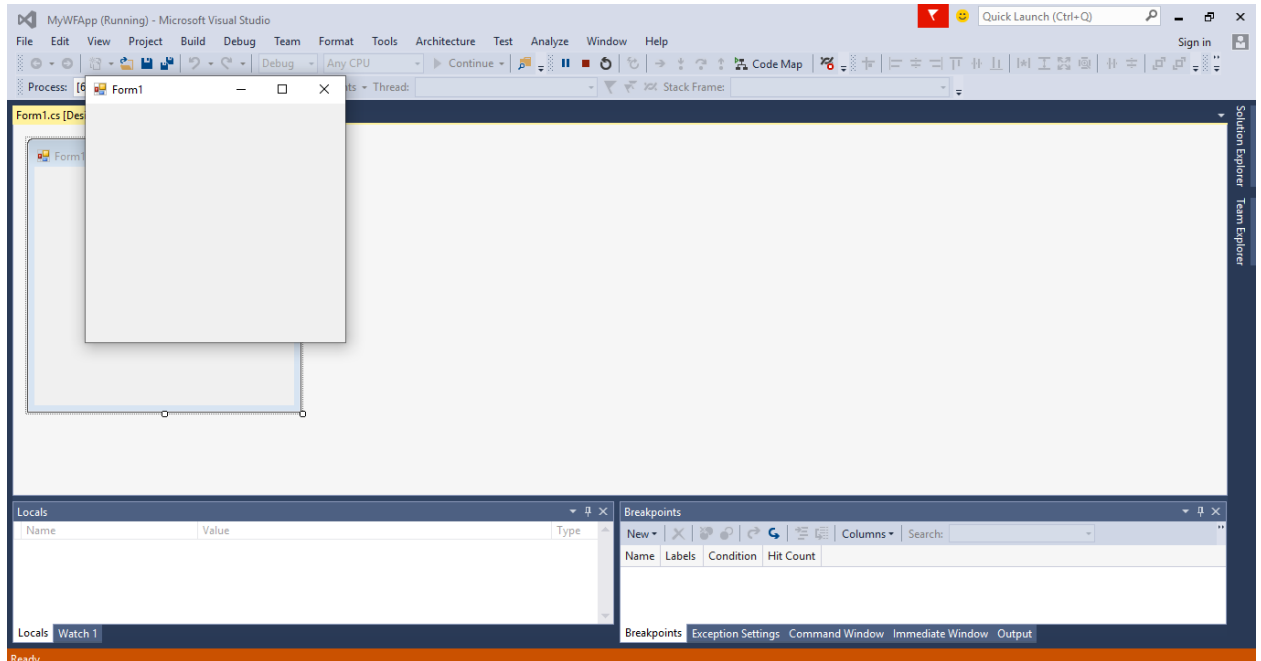


Step 5. As you can see on the windows above **Server Explorer** and **Toolbox** are stacked hidden on the left panel. Therefore, click on each of them, especially the **Toolbox** to drag and drop different control for designing a required GUI. Click on **Auto Hide** button on the **Toolbox** to let the **Toolbox** stayed displayed. As you can see on the windows shown below Toolbox contains set of categories of Windows Forms controls which we can drag and drop on to the designing area to design a GUI.



Congratulations! You have created a Windows Application

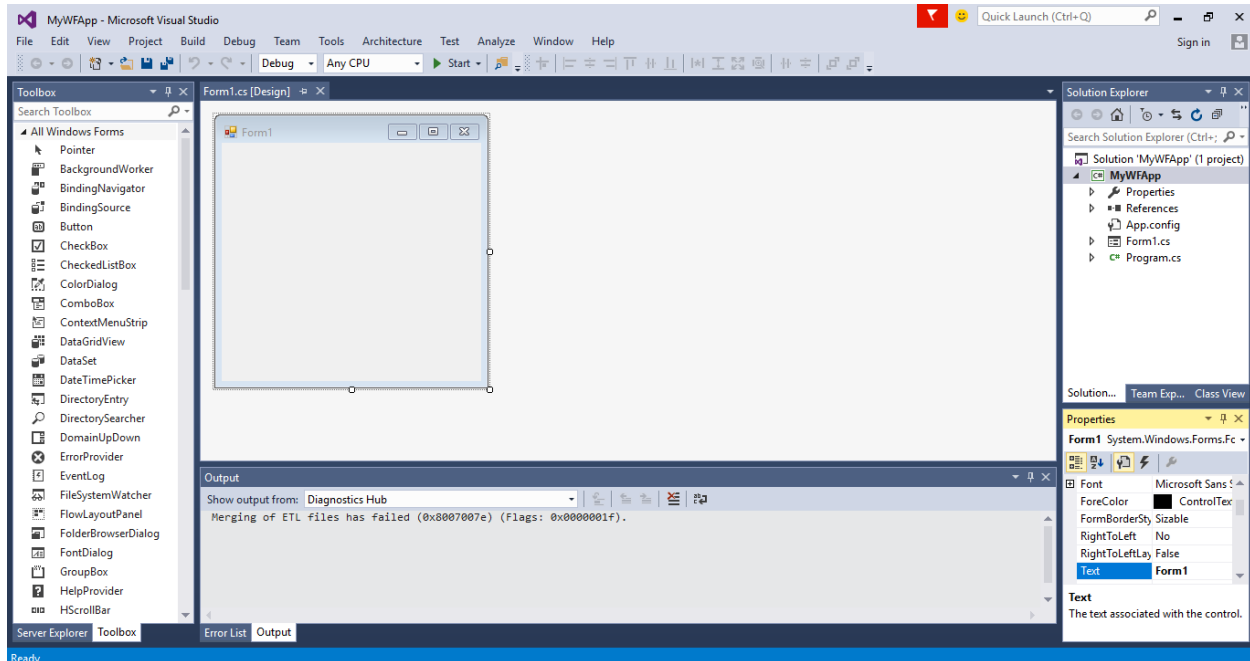
Step 6. Your project is now ready to be compiled and executed. To compile the project either go to Build=>Build Solution or press Ctrl+Shift+B execute the project. To execute the project either go to Debug=>Debugging Solution or press F5. The initial output will look like the one shown below.



After this, you can design the required GUIs based on your application requirements. Let's do simple modifications on the initial form, Form1, seen on the above screen.

- ✓ Change the caption of the form to “**Simple Calculator**”
- ✓ Change width and height of the form to **400** and **500** respectively
- ✓ Remove minimize and maximize boxes
- ✓ Remove icon of the form

To do these all changes first make sure that your project is not in running mode. If your project is in running mode close the output form to return to the project and click on **Form1** then go to **Properties** panel on the right-bottom just below **Solution Explorer**.



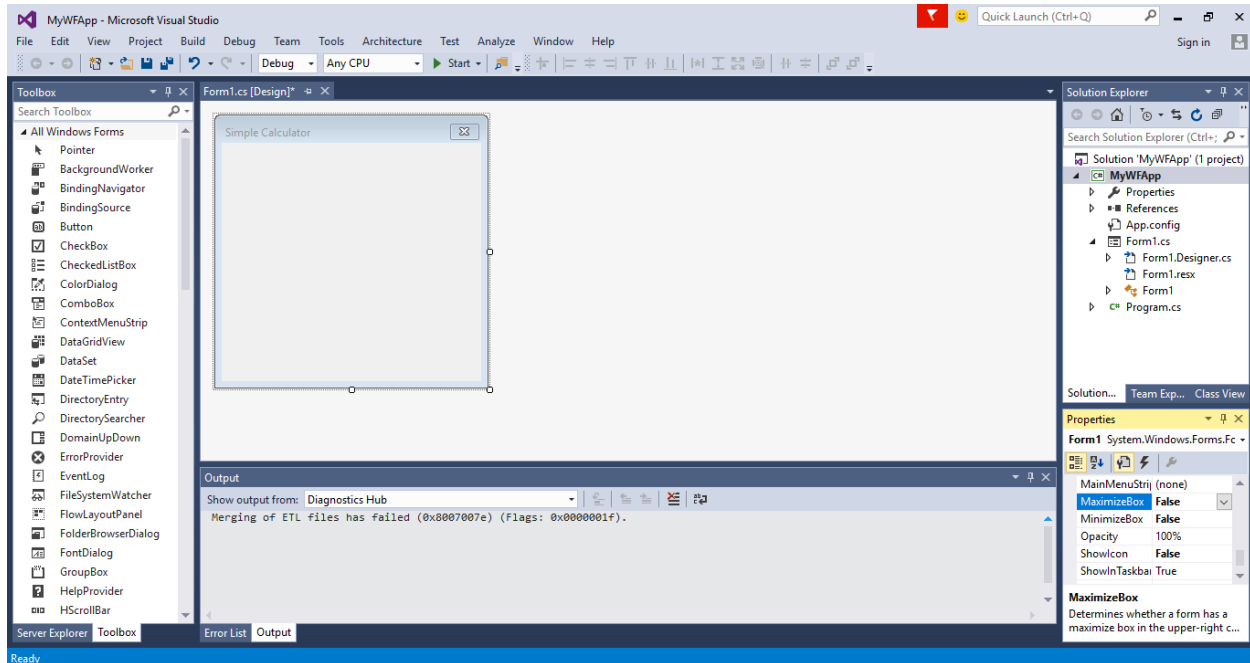
To change Caption of the form from Form1 to “**Simple Calculator**”, from **Properties** replace the value of **Text** property to “**Simple Calculator**”.

Similarly, to change the size of the form, go to **Properties** and change the value of **Size** property to 400, 500 (the default value is width=300, height=300)

To remove minimize box and maximize box go to **Properties** and change the **MinimizeBox** property value to **False** and **MaximizeBox** property value to **False**(by default both of them are **True**)

Similarly, to remove the icon of the form go to **Properties** and change **ShowIcon** property value to **False**(by default it is **True**)

NB: these all changes done through properties window can also be done programmatically in side form load event to take effect at run time.

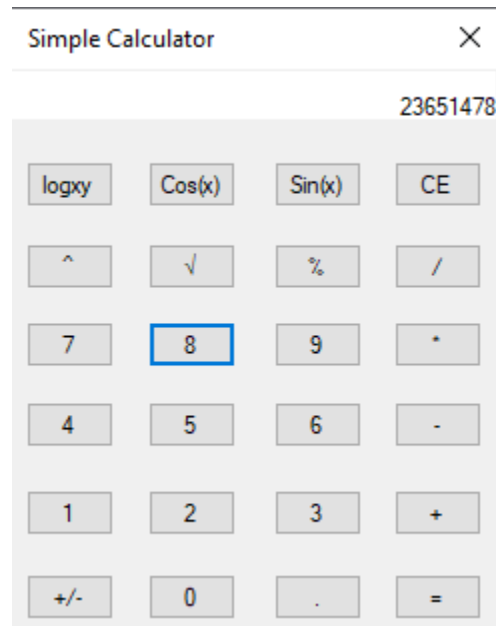


Exercise on Windows Application

In this section you will use your windows application project you created in previous section to practice the given exercises. Therefore, don't forget to open and make ready your windows application project before diving into the exercises.

1. Recall your exercise 4 on Console Application in which you created a class named **SimpleCalculator** and design a necessary GUI that will help a user to perform similar operations through GUI rather than command line interface. To design the necessary GUI:
 - a. Add new windows form to the project. To do this, right-click on the name of the project go to **New=>Windows** Form and set the **Name** of the form **frmSimpleCalc** and once the form is added to the project click on the form and go to **Properties** and change **Text** of the form from **frmSimpleCalc** to **Simple Calculator** respectively.
 - b. Drag and drop necessary controls onto the form for corresponding each member data or member method of the class. For example, for **addition** method you should drag and drop a **Button** from Toolbox onto the form and change the **Text** and **Name** property of the button from button1 to '+' and **btnAdd** respectively.
 - c. Follow the naming convention to name each control you dragged and dropped. This name will be used to call each control in code behind the controls.

The expected GUI is as shown below



- d. Write the necessary code behind each control to make the GUI functional. For example, when the numeric button 6 is pressed the number 6 should be displayed on the input textbox. Therefore, the code behind the btn6 when clicked is:

```
private void btn6_Click(object sender, EventArgs e)
{
    txtInput.Text += 6;
}
```

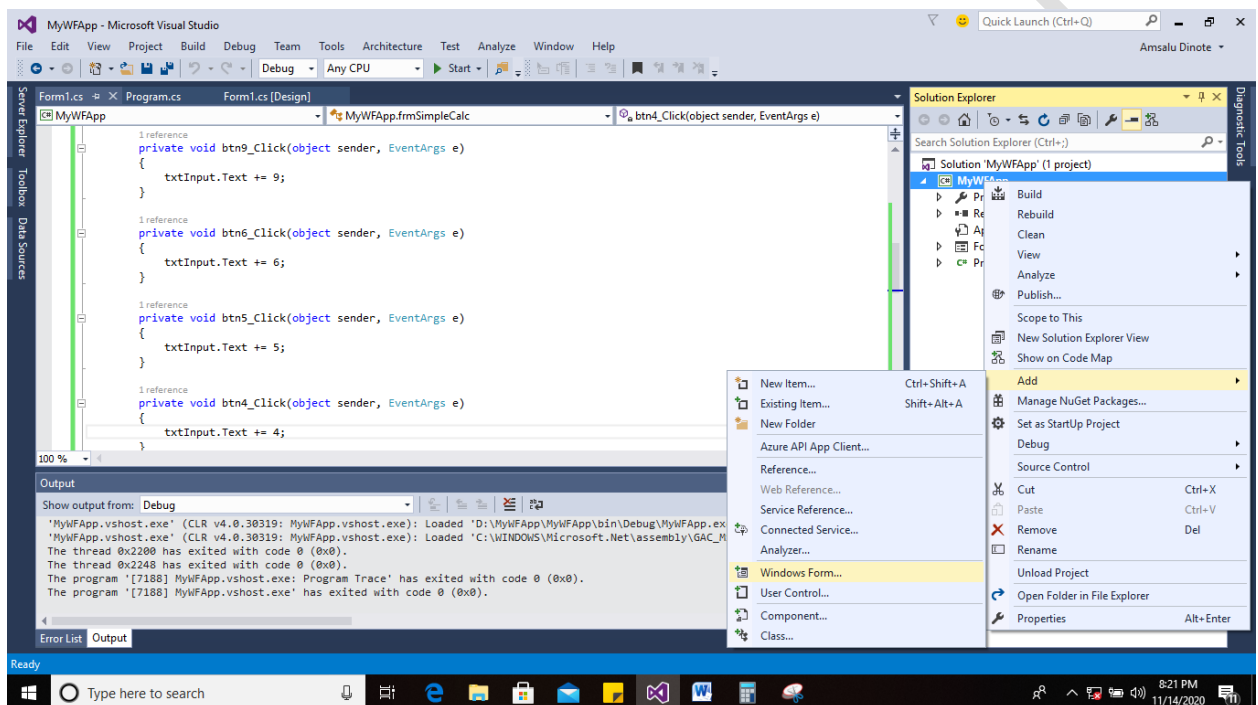
NB: to write a code behind a given control double click on the control, this action will open a method with an empty body and by the name you given for the control. Therefore, you will use that body space to write the required action(s) to be executed when the control is clicked or pressed.

- e. Compile and run the project to see the output of your design, the GUI.

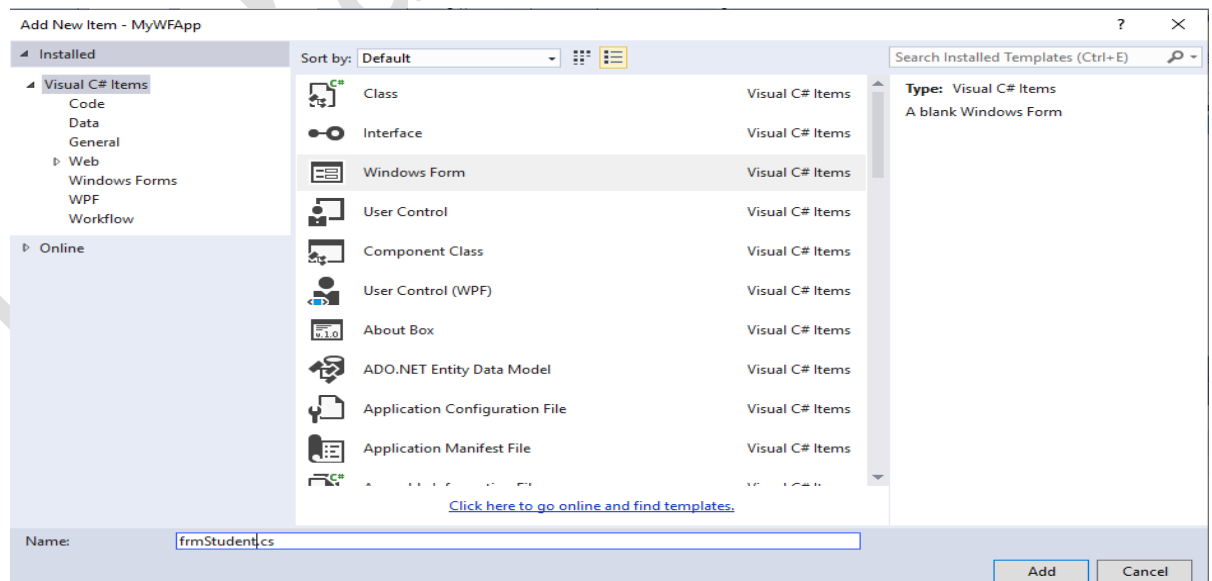
NB:-Remember that to get executed your current form you have to set the form as the starting form inside the file named **program.cs**. That is change the name **Form1** by the name of the form you want to execute inside the line:

```
Application.Run(new Form1());
```

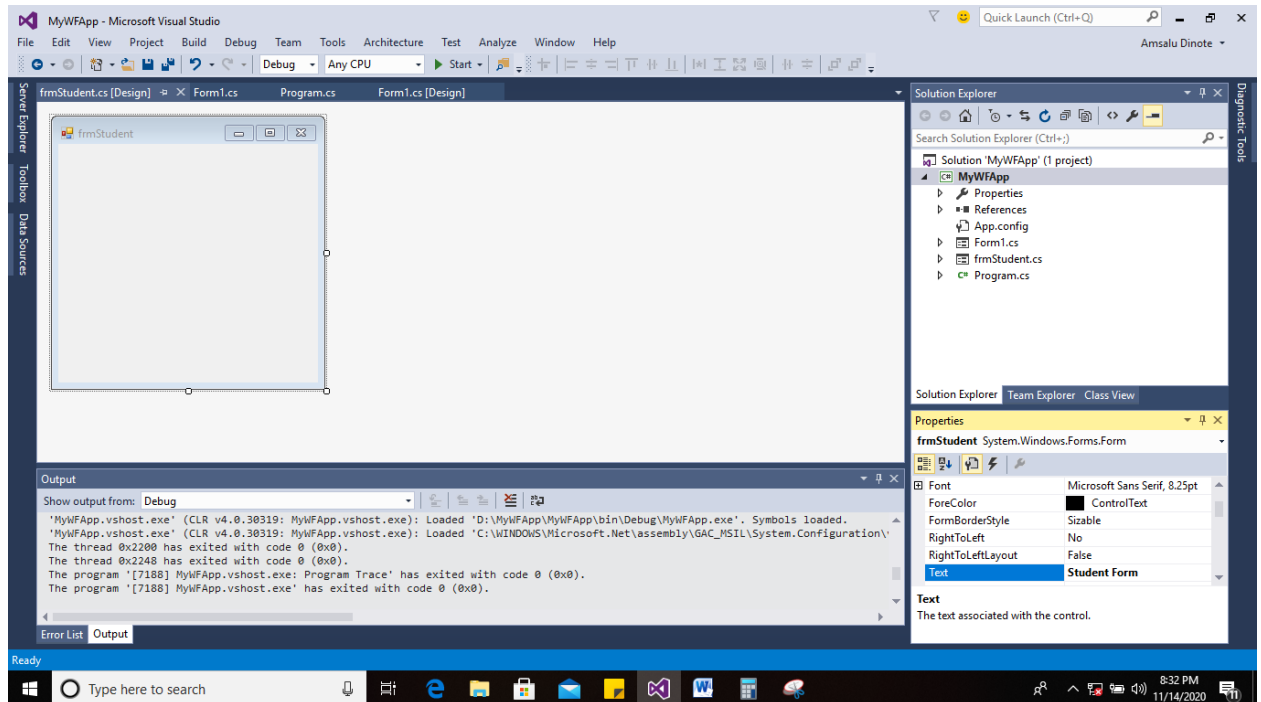
2. Recall your **Exercise 5** of Console application exercises in which you created a class named **Student**. Create a GUI with all necessary controls that will allow a user to input values of each data member. To accomplish this exercise:
 - a. Add a new windows form, right-click on the name of the project go to Add=>Winows Form...(for more refer to the screen below)



- b. Then set the **Name** of the form to **frmStudent** on the Add New Item window and press on Add button.



- c. Once the form is added to your project click on the form and go to **Properties** widow(just below Solution Explorer) and set **Text** of the form to **Student Form**



- d. Drag and drop necessary controls corresponding to each data and method member of the class from Toolbox. For example, for **name** data member you should drag and drop **Label** and **Textbox** controls onto the **Student Form** and change the **Text** of the label to **Full Name**. Don't forget to adjust the tab-order for each control. Tab-order is nothing but the number indicating the order by which the control can be accessed when a user is using tab key on the keyboard. The expected GUI is shown below.

A screenshot of the 'Student Form' GUI. It contains several input fields and dropdown menus. The fields are labeled 'ID Number', 'Full Name', 'Sex', 'Age', 'Department', and 'Program'. The 'Full Name' field is currently selected. At the bottom, there are 'Save' and 'Cancel' buttons. The form has a standard Windows window title bar with 'Student Form' and standard minimize, maximize, and close buttons.

- e. Change the default name of each control to conventionally appropriate name. For example, the default name **TextBox1** will be given for the first **TextBox** you dragged and dropped onto the form. Therefore, if this textbox is used to accept a name of student from the user, it is appropriate to name it as **txtFullName**.
- f. Compile and execute the project to see the output of your design work. Don't forget to set the form to be displayed first inside the file **Program.cs**, just as you did in the above exercise, in case you can't see display the required student form.
- g. Write a code behind the form to make the first item of each comboBox get selected on form load event. Similarly, write a code behind **Save** button so that when a user clicks on the button the message "**Data save successfully!**" is displayed if all fields are filled, otherwise displays the message "**Please, fill all fields!**". The message "**You are not eligible for registration!**" should also be displayed if the age of the student is less than 15.

The screenshot shows two windows from a Windows application. The main window, titled "Student Form", contains several input fields and buttons. The fields are: "ID Number" with the value "cs/008/08", "Full Name" with the value "f g b h g i g j h", "Sex" with a dropdown menu showing "Male", "Age" with the value "14", "Department" with a dropdown menu showing "Computer Science", and "Program" with a dropdown menu showing "Regular". Below these fields are two buttons: "Save" and "Cancel". Overlaid on the bottom right of the main window is a smaller dialog box titled "Not Eligible". This dialog box contains a yellow warning triangle icon and the text "Sorry, you are not eligible for registration!". At the bottom of this dialog box are two buttons: "Retry" and "Cancel".

- h. Add a Reset button to reset the controls to respective initial states. Then write a code behind Reset and Cancel button to reset controls and close the form respectively.
3. Modify your design of the **Student**(the one you designed in exercise 2 above) form by doing the following tasks
- a. Drag and drop **TabControl** from **Toolbox** onto the **Student** form.
 - b. Change the **Text** of the first page of the **TabControl** to **Add Student** and the **Text** of the second page to **Student List**.

- c. Move the previous data entry controls you designed in exercise 2 onto the first page, the **Add Student** page
- d. Drag and drop **ListView** control from Toolbox onto page 2, the **Student List** page and name the control as **lvStudentList**
- e. Write code behind the controls to receive student inputs from user and display the added students on the **ListView**.

This can be done by modifying the code inside the **btnSave** method. Add the lines of code that adds the student data to listview control. Take look at the following code.

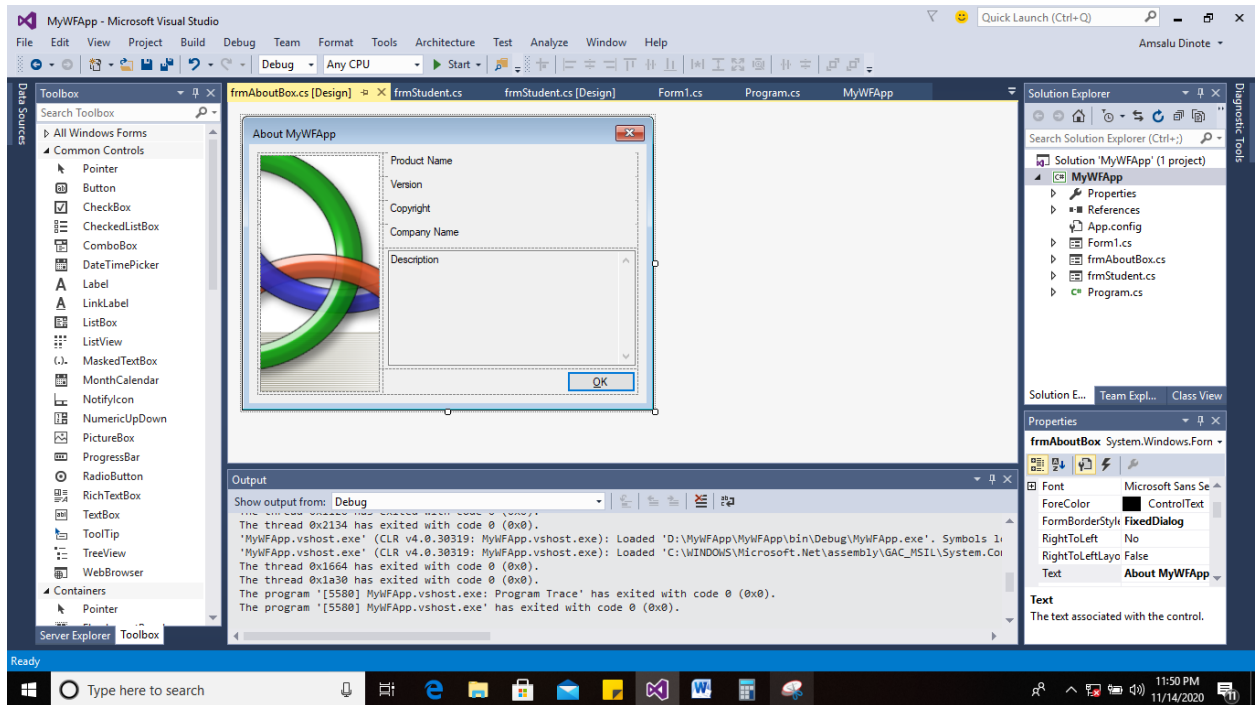
The modified GUI after application of the above actions will look like the following.

The screenshot shows the 'Student Form' application with two tabs: 'Add Student' and 'Student List'. The 'Add Student' tab is active, displaying input fields for ID Number (cs/006/08), Age (32), Full Name (Aster Adane Abebe), Department (Computer Science), Sex (Female), and Program (Regular). Below the fields are 'Save', 'Reset', and 'Cancel' buttons. A 'Successful' dialog box is overlaid on the right, displaying 'Student Data Saved Successfully!' and an 'OK' button.

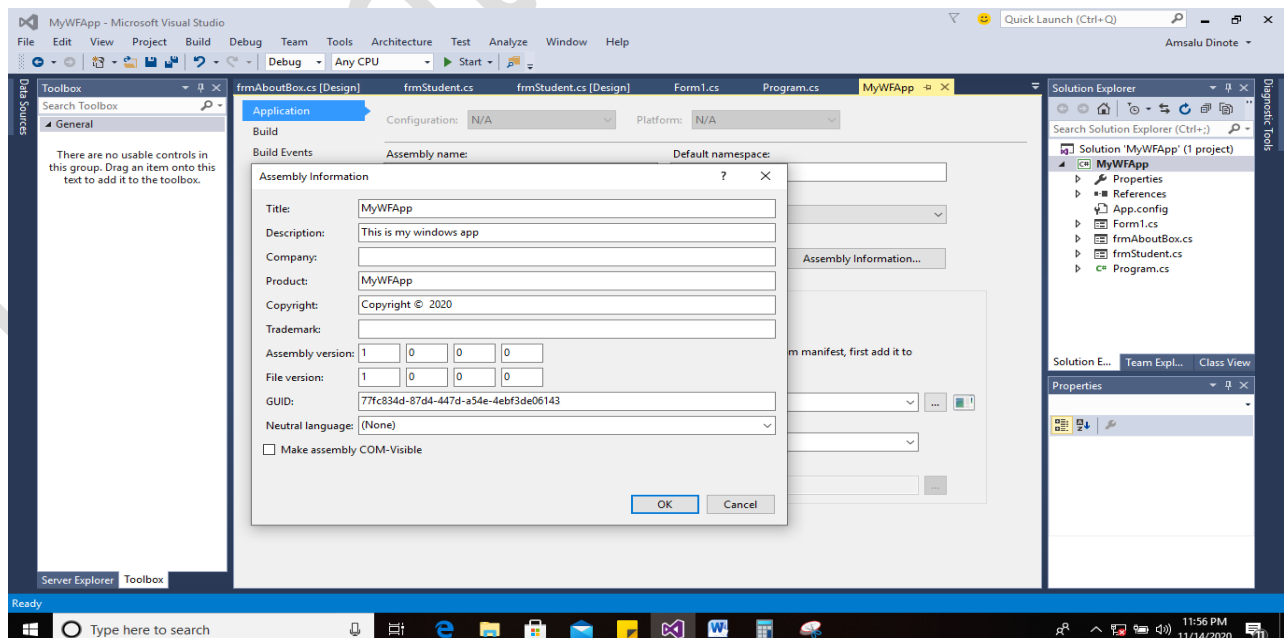
The screenshot shows the 'Student Form' application with the 'Student List' tab active. It displays a table with the following data:

Student ID	Full Name	Sex	Age	Department	Program
cs/006/08	Aster Adane Abebe	Female	32	Computer Sc...	Regular

4. Add **AboutBox** to your project and write necessary discretion to be displayed on the box. This can be done by right-clicking on the project then **Add=>New Item...**, Select **AboutBox** and set the name of the form as **frmAboutBox.cs** The following is the AboutBox.



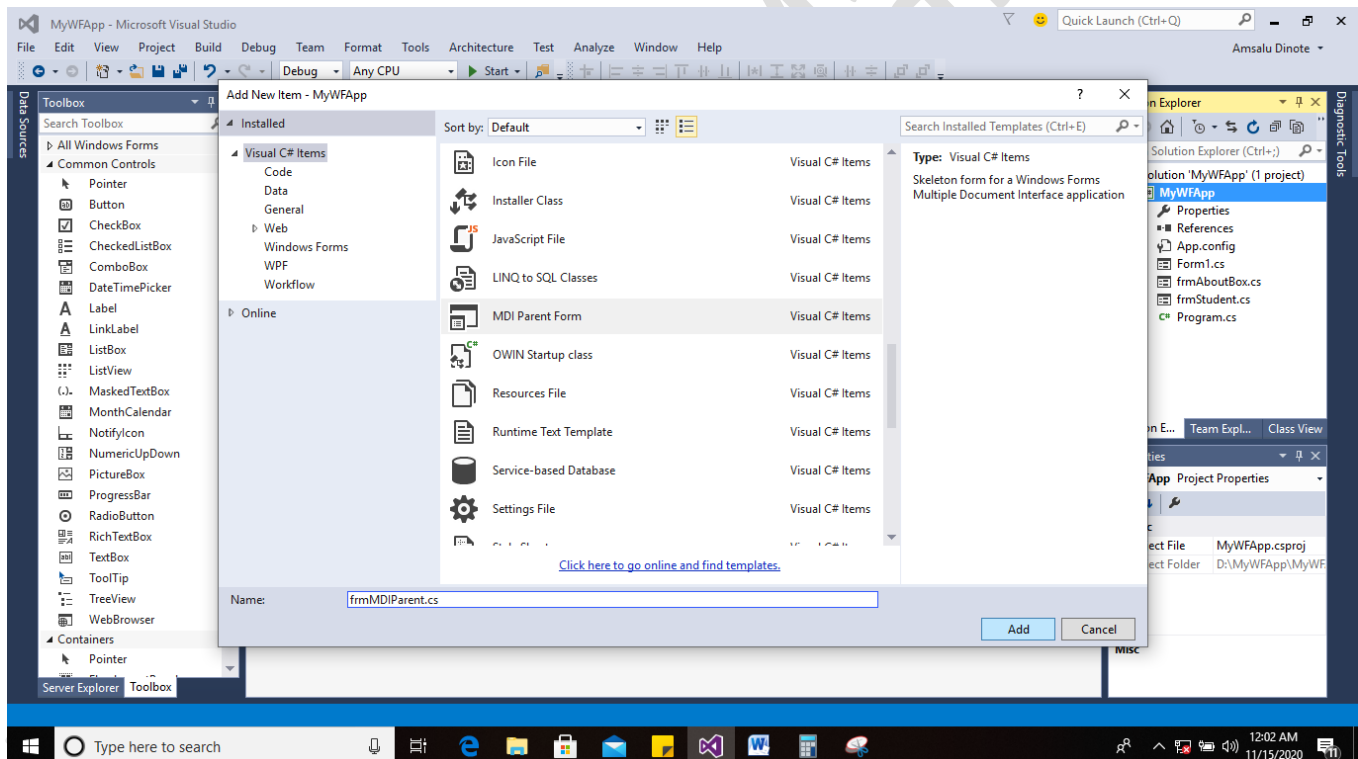
The details to be displayed on this box will be edited by following: Right-click on Project name then go to **Properties...** then a window with the name of your project will be displayed. On that window, go to **Application** (left panel)=>click on **Assembly Information...** button, this will



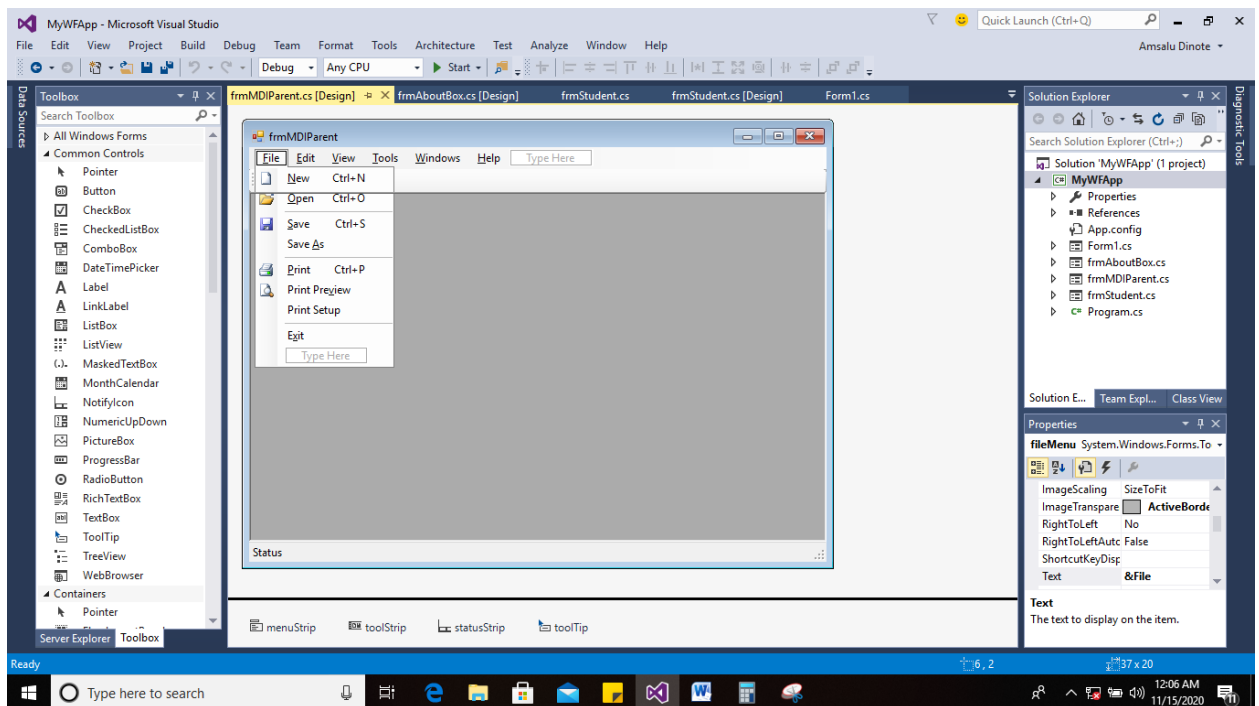
display the window shown above, from which you can edit the details. These details will be automatically retrieved and displayed on the **AboutBox** at run time.

5. Assume that you need to access all your project forms from single form without closing the running application and changing a form to be displayed first. The appropriate solution to case is to use a type of form named MDI (stands for Multiple Document Interface) form. MDI is a form by default with Menu bar, Icon bar, and Status bar. The menu bar contains menu items used to access other forms. The icon bar contains different small icons used to provide a shortcut access to some forms of an application. Status bar is used to display some status of current actions such as a name of currently displayed form. To use this MDI form

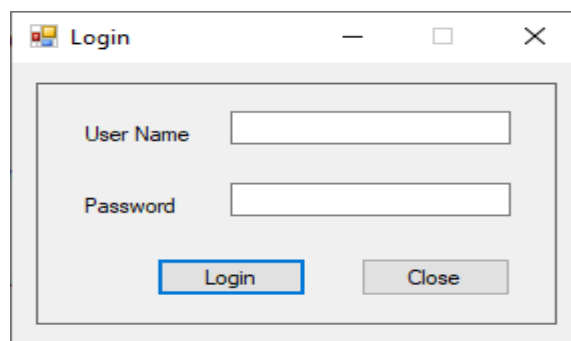
- a. Right-click on the project go to Add => New Item => MDI Parent Form, set the name of the form as *frmMdi.cs*



- b. Once the form is added to your project, set the appropriate menu items text and shortcuts (e.g. Ctrl+O to open Student Form) to corresponding forms to be access through the MDI.
- c. On the MDI form go to File Menu then File Menu Item and go to **Properties** and change Text of menu item to Add Student.



- d. To access **Student Form** from MDI go to **Add Student** File Menu Item and double click on it this will display an empty-body delegate corresponding to the click event of the **Add Student** Menu Item.
- e. Write a code that will show the form **frmStudent**. Do the same for displaying **frmSimpleCalc** and **frmAboutBox** forms.
6. Assume that you want to secure your application so that only authenticated users will get into your Web application. This done by adding Login From to your project.
 - a. Right-click on your project go to **Add => New Item...**, select Window Form and set name of the form **frmLogin.cs**
 - b. After the form is added to your project adjust the form to appropriate size and appearance then add labels and textboxes for **User Name** and **Password** inputs.



- c. Hardcode a user name and password into the code behind the login button to check the user credential against. This is because for this time we don't have a database for our application, later on, we do.
 - d. To let the application start execution from Login Form change the Run method parameter to **frmLogin** inside **Program.cs** file. To do this, go to Solution Explorer then click on the file **program.cs** to open its contents as shown below.
7. Design the following GUI to and write necessary code behind the GUI

The screenshot shows a Windows Form titled "Window Programming" with three tabs: "Registration", "Data Edit", and "Data View". The "Registration" tab is selected. It contains two main sections: "Course Data" and "Student Data".

Course Data:

- Course Title:
- Course Code:
- Cr. Hr.:

Student Data:

- ID No.:
- Full Name:
- Gender: ☐ Male ☐ Female
- Age:
- Reg. Date:
- Interest:
 - ☒ Web Design
 - ☐ Windows Programming
 - ☐ Mobile App Development
 - ☐ Compiler Design
 - ☐ Multimedia Programming

Below the Student Data section is a "Save/Cancel" group box containing "Save" and "Cancel" buttons.

On the left side of the Student Data section, there is a calendar for April 2018. The date 23 is selected, and the text "Today: 4/23/2018" is displayed below the calendar.

While you are designing the GUI accomplish the following tasks:

- ✓ Arrange a tab order for all controls
- ✓ Add a tooltip for all controls, for example, for PictureBox the tool tip should display the message "**Click to Add Photo**"
- ✓ From property window set all important features for all controls. For example, for the form set width and height respectively to 719 and 396, form Text to "**Window Programming**", **MaximizeBox** to **False**, **FormBorderStyle** to **FixedSingle**, etc.
- ✓ When form opens the Gender radio button male should be selected by default. And interest of the student **Web Design** should be checked by default as well.
- ✓ When clicking on Cancel button it should reset all form controls to their initial values. That is, all text boxes to empty, radio buttons and checkboxes unselected
- ✓ If save button is clicked
 - the form should be checked for existence of empty text fields
 - credit hour of a course should not be text and not exceed 6
 - only students having age greater than 20 and less than 35 can be registered and error message should be displayed otherwise
 - for male students their interest should not exceed 2